Git 及 GitHub 使用心得体会

张新越

2026.10.25

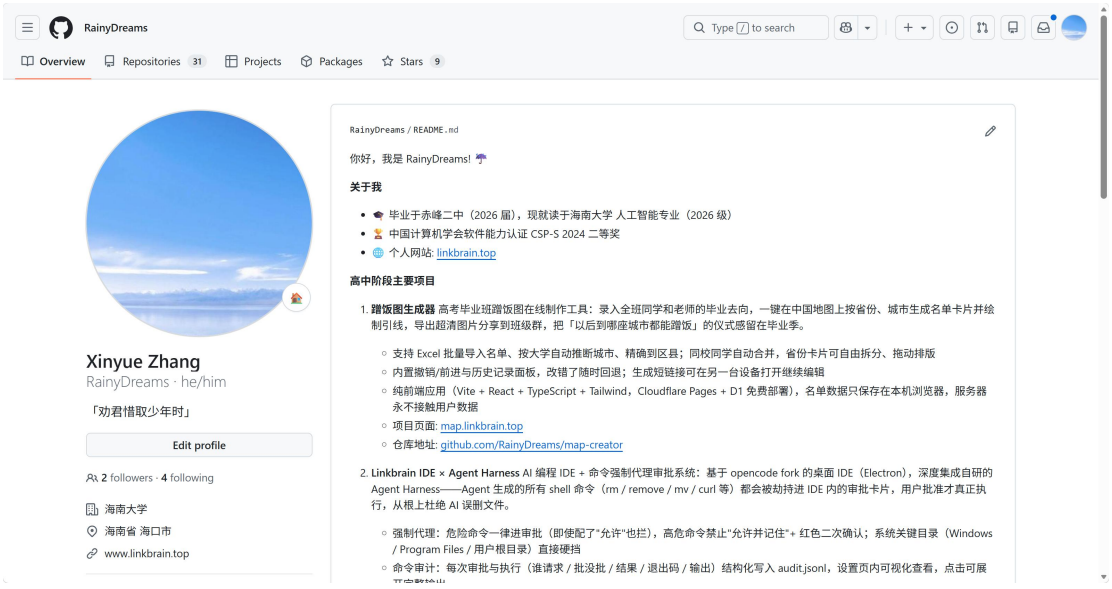
之前高中就接触过 Git 的基础操作，也试着参与过一些开源项目，这次跟着一步步完成仓库 Fork、克隆、新建分支提交作业的整套流程，再一次实实在在感受到版本管理工具在开发中的分量。

以前写代码(在我第一次知道 git 之前)，图省事经常直接复制文件，命名随便写： abc.cpp 这类文件一大堆，甚至于找 zip 打包做版本管理，改来改去自己都分不清哪个版本是能用的，一旦改错内容、删掉一段逻辑，想找回之前可用的代码特别麻烦。这次实操下来我更确定一个想法：编程里面，文件命名规范和版本追踪，是开发工作里很基础的一环。好的命名和版本管理既能让别人，也让之后的自己一眼看懂文件用途。例如 commit 的命名的好能让人一言就知道这次更新干啥的，当然有了 Agent 之后 commit -m 的提交省事了很多，它的摘要写的很好。

分支机制是我这次体会很深的一点。main 或者 master 分支用来存放稳定、可运行的代码，我们单独新建 homework 分支在上面写作业、做修改，不用担心改动直接进入主仓库的内容。放到团队协作场景里，大家各自在独立分支上（例如 fix-某某）开发功能、调试代码，写完再合并，既能并行推进工作，~~又不容易互相打乱别人的进度（可能会有合并冲突）~~，这也是我觉得为什么现在~~几乎~~所有开发都离不开 Git！

版本管理的思想，本质上就是帮我们有序留存每一步尝试，区分稳定版本和正在尝试修改的内容。

以后自己做项目、参与开源或者后续科研相关的工作，我也会继续养成规范管理文件、用好分支的习惯。




```
Microsoft Windows [版本 10.0.26300.9711]
(c) Microsoft Corporation。保留所有权利。

D:\Learn\RoboMaster>git clone https://github.com/RainyDreams/shijuepeixun.git
Cloning into 'shijuepeixun'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

D:\Learn\RoboMaster>cd shijuepeixun

D:\Learn\RoboMaster\shijuepeixun>git checkout -b homework/zhangxinyue
Switched to a new branch 'homework/zhangxinyue'

D:\Learn\RoboMaster\shijuepeixun>git add .
```